

Dynamic Activation Functions in Neural Networks

Salveen Dutt

May 2026

1 Introduction

Standard feedforward neural networks rely on fixed activation functions—ReLU, Sigmoid, Tanh—whose shapes are chosen by the practitioner before training and remain constant throughout. This project investigates whether equipping each hidden neuron with a small set of *learnable* parameters that govern its activation shape can yield accuracy improvements or convergence speedups beyond what fixed activations achieve.

Two hypotheses are tested:

1. Given a network already trained with fixed activations, freezing the weights and fine-tuning only the per-neuron activation parameters recovers accuracy that the fixed shape left on the table.
2. Activation parameters learned atop one set of trained weights transfer to fresh random weights and accelerate convergence—evidence that a task-optimal activation shape exists independently of the specific weight initialisation.

2 Methodology

2.1 Architecture

All experiments use the same two-hidden-layer MLP:

$$\text{Input} \xrightarrow{W_1, b_1} \mathbb{R}^{256} \xrightarrow{\text{act}} \xrightarrow{W_2, b_2} \mathbb{R}^{128} \xrightarrow{\text{act}} \xrightarrow{W_3, b_3} \mathbb{R}^{10} \xrightarrow{\text{softmax}} \hat{y}$$

Table 1: Fixed hyperparameters shared across all experiments

Component	Setting
Hidden dimensions	256, 128
Output	10-class softmax
Loss	Cross-entropy
Optimiser	Mini-batch SGD
Learning rate	0.01
Batch size	128
Max epochs	30 (per phase)
Early stopping	Patience = 10
Weight initialisation	He (ReLU-based), Xavier (Sigmoid-based)
Random seeds	20 per experiment

2.2 Datasets

Table 2: Datasets used across all experiments

Dataset	Input dim	Train / Val split	Classes
MNIST	784	56,000 / 14,000	10
Fashion-MNIST	784	56,000 / 14,000	10
CIFAR-10	3,072	48,000 / 12,000	10

All features are standardised via `StandardScaler` fitted on the training split.

2.3 Dynamic Activation Functions

Each dynamic activation introduces two learnable scalar parameters (a_j, b_j) *per neuron* j . Gradients for these parameters are derived analytically and computed during backpropagation.

Table 3: Learnable activation function definitions

Activation	Definition	Parameters & Init
DynamicReLU	$f(x) = \max(a_j, b_j \cdot x)$	$a_j = 0, b_j = 1 \Rightarrow$ reduces to ReLU
DynamicSigmoid	$f(x) = \frac{1}{1 + e^{-b_j(x-a_j)}}$	$a_j = 0, b_j = 1 \Rightarrow$ reduces to Sigmoid
ReLU-Sigmoid Blend	$f(x) = a_j \cdot \text{ReLU}(x) + b_j \cdot \sigma(x)$	$a_j = 0.5, b_j = 0.5$

The initialisation conditions ensure that each dynamic activation starts as its fixed-function counterpart; the standard activation is always a reachable special case, so finetuning can only improve accuracy in principle. Gradients with respect to (a_j, b_j) for the blend are:

$$\frac{\partial f}{\partial a_j} = \text{ReLU}(x), \quad \frac{\partial f}{\partial b_j} = \sigma(x)$$

2.4 Parameter Budget

The network has $\approx 200,000$ weight parameters on MNIST (784-input) and $\approx 820,000$ on CIFAR-10 (3,072-input). Dynamic activations add $2 \times (256 + 128) = \mathbf{768}$ parameters—less than 0.4 % of the MNIST weight count, and less than 0.1 % for CIFAR-10. The central question is whether this thin slice captures degrees of freedom that weights alone cannot efficiently reach.

2.5 Experimental Protocols

Experiments 1–3 (Activation Finetuning). Each experiment follows a two-phase protocol:

1. *Baseline:* Train the MLP with fixed activations to convergence (up to 30 epochs, early stopping).
2. *Finetuning:* Copy the converged weights into a new model with dynamic activations (initialised to match the fixed baseline). Freeze all weight parameters; train only (a_j, b_j) for up to 30 additional epochs.

Statistical significance is assessed using paired t -tests across 20 seeds ($\alpha = 0.05$) with Cohen’s d for effect size.

Experiment 4 (Pretrained Activation Transfer). A three-phase protocol:

1. Train baseline ReLU MLP $\rightarrow$ converged weights W^* .
2. Freeze W^* ; learn DynamicReLU parameters (a^*, b^*) .
3. Reinitialise weights randomly; freeze (a^*, b^*) ; train only the weights. Compare convergence speed and final accuracy with Phase 1.

3 Results

3.1 Experiment 1 — Dynamic ReLU Finetuning

DynamicReLU finetuning produces statistically significant improvements on Fashion-MNIST and CIFAR-10, with no change on MNIST. The absolute gains are small but consistent across seeds.

Table 4: Baseline ReLU vs. DynamicReLU after activation finetuning (mean $\pm$ std, $n = 20$)

Dataset	Baseline	+DynamicReLU	Δ Acc	Significant?
MNIST	0.9639 ± 0.0006	0.9639 ± 0.0007	+0.00%	No ($p = 0.84$)
Fashion-MNIST	0.8845 ± 0.0020	0.8858 ± 0.0016	+0.13%	Yes ($p < 0.0001$)
CIFAR-10	0.5155 ± 0.0038	0.5201 ± 0.0030	+0.46%	Yes ($p < 0.0001$)

Table 5: Baseline ReLU vs. ReLU–Sigmoid blend after finetuning (mean $\pm$ std, $n = 20$)

Dataset	Baseline	+Blend	Δ Acc	Significant?
MNIST	0.9639 ± 0.0006	0.9593 ± 0.0010	−0.46%	Yes ($p < 0.0001$, worse)
Fashion-MNIST	0.8845 ± 0.0020	0.8814 ± 0.0019	−0.31%	Yes ($p < 0.0001$, worse)
CIFAR-10	0.5155 ± 0.0038	0.5213 ± 0.0036	+0.58%	Yes ($p < 0.0001$, $d = 1.52$)

3.2 Experiment 2 — ReLU–Sigmoid Blend Finetuning

The blend yields the largest single improvement (+0.58% on CIFAR-10, large effect $d = 1.52$) but simultaneously *degrades* performance on MNIST and Fashion-MNIST. Introducing sigmoid nonlinearity disrupts the already well-adapted ReLU representations on simpler datasets while providing useful additional expressiveness on the harder CIFAR-10 task.

3.3 Experiment 3 — Dynamic Sigmoid Finetuning

Table 6: Baseline Sigmoid vs. DynamicSigmoid after finetuning (mean $\pm$ std, $n = 20$)

Dataset	Baseline	+DynamicSigmoid	Δ Acc	Significant?
MNIST	0.9128 ± 0.0009	0.9131 ± 0.0010	+0.03%	No
Fashion-MNIST	0.8339 ± 0.0012	0.8346 ± 0.0013	+0.07%	No
CIFAR-10	0.4189 ± 0.0019	0.4197 ± 0.0019	+0.08%	No

Improvements are negligible and statistically insignificant across all datasets. The learned (a_j, b_j) parameters remained close to their initialisations ($a \approx 0$, $b \approx 1$), suggesting that the sigmoid shape is already near-optimal after weight training and the added degrees of freedom are not exploited.

3.4 Experiment 4 — Pretrained Activation Transfer

Pretrained activation parameters provide neither a meaningful accuracy improvement nor a convergence speedup. Paired t -tests confirm no statistically significant differences at $p < 0.05$. Post-hoc inspection reveals that the learned (a^*, b^*) parameters deviated only marginally from their ReLU initialisations ($a \approx 0$, $b \approx 1$), explaining the inability to create a qualitatively distinct loss landscape for new weights.

4 Analysis & Discussion

4.1 Statistical vs. Practical Significance

Experiments 1 and 2 both yield $p < 0.0001$ on CIFAR-10, yet the absolute gains (+0.46%, +0.58%) are small. The high statistical power stems from 20-seed paired testing, which is sensitive enough to detect even negligible differences. The effect sizes—moderate for DynamicReLU, large for the blend on CIFAR-10 ($d = 1.52$)—are genuine but not practically meaningful given the absolute scale.

4.2 The Parameter Ratio Argument

The weight matrices already provide sufficient degrees of freedom to approximate the target function. With $< 0.4\%$ as many activation parameters as weights, the optimisation landscape is dominated by the weight

Table 7: Phase 1 (fixed ReLU) vs. Phase 3 (pretrained activations, fresh weights)

Dataset	Phase 1 Acc	Phase 3 Acc	Δ Acc	Epochs saved
MNIST	0.9639 ± 0.0006 (30 ep)	0.9640 ± 0.0006 (30 ep)	+0.01%	0.0
Fashion-MNIST	0.8845 ± 0.0020 (30 ep)	0.8847 ± 0.0022 (30 ep)	+0.02%	0.0
CIFAR-10	0.5155 ± 0.0038 (21.6 ep)	0.5147 ± 0.0033 (21.1 ep)	-0.08%	0.5

subspace. Any representation achievable by adjusting (a_j, b_j) can be approximately reproduced by re-scaling the adjacent weight rows—the network does not need the activation parameters to compensate.

4.3 Dataset Complexity

A consistent pattern emerges: dynamic activations help more on harder datasets (CIFAR-10) than on easier ones (MNIST). This aligns with the capacity-constraint hypothesis: when a fixed activation is already near-optimal for a simple task (MNIST with ReLU), learnable shape adds noise rather than signal. On CIFAR-10, where the fixed baseline is weaker (51%), there is more room for the activation shape to matter.

4.4 Why Transfer Does Not Work

The failure of Experiment 4 is informative. The learned activation parameters stayed close to the ReLU defaults, meaning Phase 2 produced only a marginal deviation from standard ReLU. The activation and weight parameters must be *co-adapted* during training to exploit their joint degrees of freedom; learning them sequentially (weights first, activations second, then transfer) breaks this co-adaptation and leaves the activations carrying almost no task-specific information.

5 Conclusion

Learnable per-neuron activation parameters do not provide practically meaningful accuracy gains or convergence speedups in two-hidden-layer MLPs trained on MNIST, Fashion-MNIST, and CIFAR-10. While some statistically significant results are observed (Dynamic ReLU and the blend on CIFAR-10), the improvements are small in magnitude and inconsistent across datasets—the ReLU-Sigmoid blend significantly *hurts* performance on simpler datasets. Activation transfer to fresh weights produces no benefit.

The root cause is the extreme parameter asymmetry: 768 activation parameters against $\sim 200,000$ to $\sim 820,000$ weight parameters. The network already has sufficient weight degrees of freedom to subsume any shape adjustment that (a_j, b_j) could provide.

Dynamic activations may offer larger benefits in **capacity-constrained regimes**: very small networks, limited training data, or tasks hard enough that the fixed activation shape is a genuine bottleneck. These directions remain open for future investigation.